Scenario: "How Do You Manage State in a Large React App?"

2 min read Updated 24 Mar 2026 Topic 45 of 62

Bookmark Mark done

10.3 – Scenario: "How Do You Manage State in a Large React App?"

🎯 Why This Question Exists

This tests whether you can think strategically about state architecture, not just pick a library.

The Framework: State Layering

Rather than choosing "Context or Redux," describe layers:

Layer 1: Local component state (useState, useReducer)

Form inputs, modals, tooltips, hover states, tab selection. Most state in any app belongs here. Rule: if only one component or its immediate children use it, keep it local.

Layer 2: Shared UI state (lift state up or Context)

Sidebar collapsed/expanded, active theme, authenticated user. Use Context for values that change infrequently (theme, auth, locale).

Layer 3: Complex global state (Redux Toolkit or Zustand)

Shopping cart with add/remove/update/clear across many pages. Notification system with multiple sources. Only use when state is truly global, transitions are complex, and you need middleware or devtools.

Layer 4: Server/async state (React Query or RTK Query)

API responses that need caching, refetching, pagination, optimistic updates. Separating server state from client state is the biggest architectural improvement you can make.

Sample Answer

> "I think of state in layers. Most state is local, using useState for simple values and useReducer for complex ones. For app-wide low-frequency values like theme and auth, I use Context with a custom hook. If the app has complex state with many transitions, I bring in Redux Toolkit. But the biggest win is separating server state from client state. I use React Query for all API data because it handles caching, background refetching, and loading states automatically. This layered approach keeps each type of state managed by the right tool."

QUICK CHECK — no marks, just for you.

1. According to the layering framework, what is the rule for deciding that a piece of state belongs in Layer 1 (local component state)?

- ○ If the value is a primitive rather than an object or array, keep it local
- ○ If only one component or its immediate children use it, keep it local
- ○ If the value changes more than once per second, keep it local for performance
- ○ If the component is a leaf node with no children, keep it local

2. The lesson recommends Context for a specific kind of shared UI state. Which characteristic makes a value a good fit for Context?

- ◯ It updates on every keystroke and must reach deeply nested components
- ◯ It requires middleware and devtools for debugging transitions
- ◯ It changes infrequently, such as theme, auth, or locale
- ◯ It comes from an API and needs caching and background refetching

Check answer

3. In this scenario answer, which single decision is presented as the biggest architectural improvement you can make in a large React app?

- ◯ Moving all Context providers into a single root provider component
- ◯ Replacing every useState call with useReducer for predictability
- ◯ Separating server state from client state
- ◯ Choosing Redux Toolkit over Zustand for global state

Check answer

React.js Interview Kit

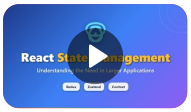
ARTICLES & DOCS

- Managing State
  react.dev
- Choosing the State Structure
  react.dev

VIDEOS



React state management
YouTube



State management: the beginner's guide
YouTube



The state of state management in React
YouTube

Hand-picked references for this lesson.